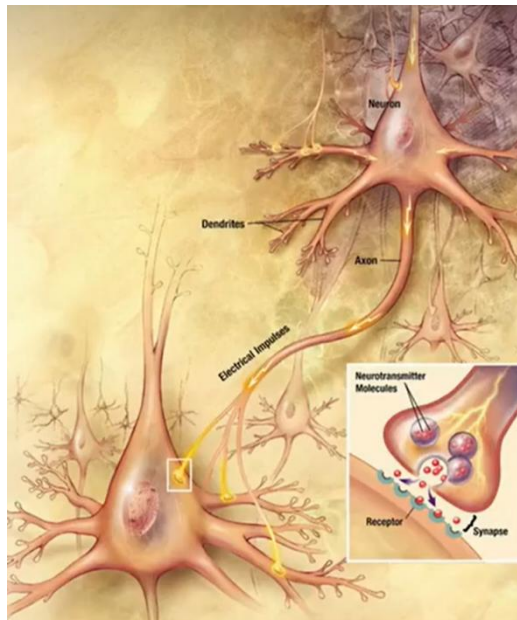


4 神经网络 (Neural Networks)

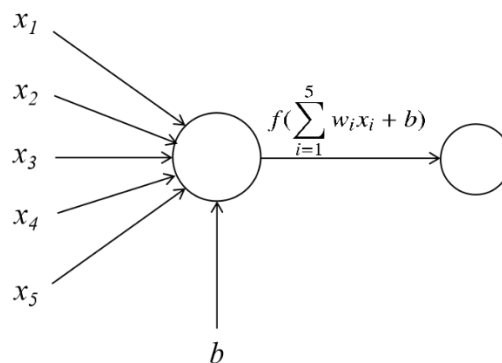
设计神经网络的最初动机是编写能够模仿人类大脑学习和思考方式的软件。现今，神经网络也被称为人工神经网络，其工作方式已经与我们所认为的大脑实际工作方式和学习方式大不相同。

研究神经网络的工作始于 20 世纪 50 年代，之后的一段时间它并不受欢迎。在 20 世纪 80 年代和 90 年代初，神经网络再次流行起来，并在一些任务中展现出很好的效果，比如手写数字识别。但是在 90 年代末，神经网络的热度再次下降，大约 2005 年开始，它又重新兴起，并被重新命名为深度学习 (Deep Learning)。从这时起，深度学习改变了很多应用领域，比如：语音识别、计算机视觉、自然语言处理等。

虽然深度学习与大脑的工作方式大不相同了，但其模仿大脑工作的动机仍然没变。因此，首先来看看大脑是如何工作的。



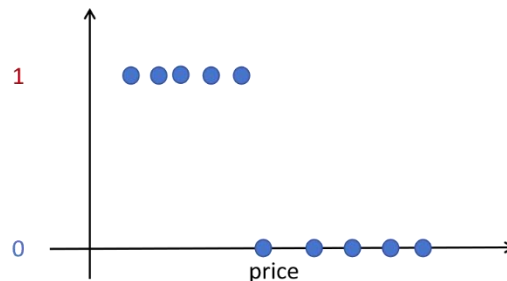
人工神经网络用一个简化的数学模型来模拟生物神经元的行为，下图为一个神经元的模型。



在构建人工神经网络或深度学习模型时，通常需要同时模拟多个这样的神经元，让这些简单的神经元相互组合连接可以构建强大的深度学习模型。

4.1 神经网络工作原理

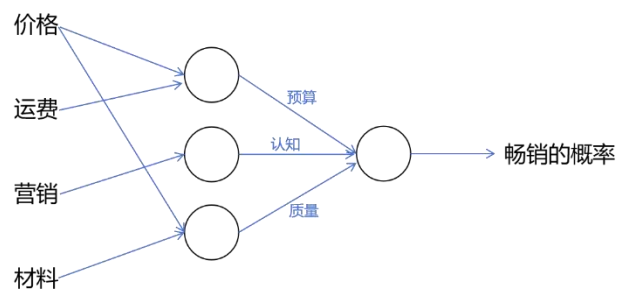
假设现在要预测一件 T 恤是否能成为畅销品，训练数据如下图所示，输入特征是 T 恤的价格，标签为 0 和 1。



如果使用 sigmoid 函数拟合数据，这是一个逻辑回归模型。为了构建神经网络，令

$$a = \frac{1}{1 + e^{-(wx+b)}}$$

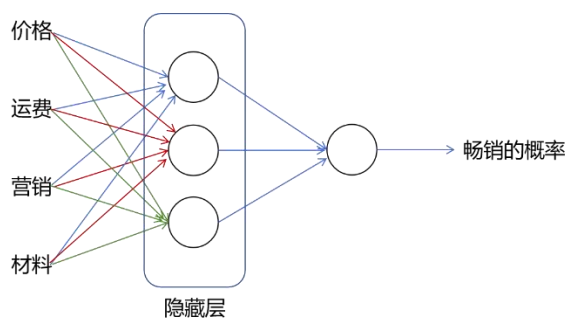
这里 a 表示激活值（Activation），激活值表示一个神经元向其下游神经元输出信号的强度。因此，一个逻辑回归模型可以看成是神经网络中一个简化的单个神经元模型。构建一个神经网络模型就是将许多这样的神经元连接在一起。如下图所示给出关于 T 恤的更多信息，用来预测 T 恤是否能成为畅销品。



神经网络能够利用原始的输入特征学习出新的特征，这些特征更有利于后续进行分类。

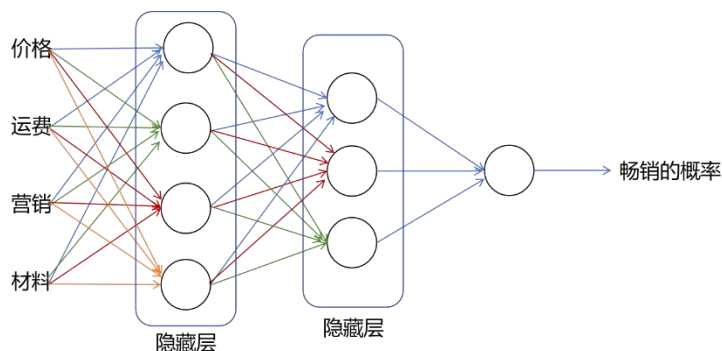
将前三个神经元组合成一层（Layer），层是一组神经元，它们以相同或相似的特征作为输入，并输出一些激活值。最后一个神经元也形成一层称为输出层，因为这一层输出的是最后的预测概率。在神经网络中，最左侧的输入特征也形成一层称为输入层，这一层只是输入特征的列表，并不做任何计算。

前面的例子需要人工地选择每一个神经元的输入特征用以学习出新的特征，如下图所示一种更简单的方式是每一个神经元都使用全部的输入特征，让神经网络自己学习出有助于分类的隐藏特征，而不需要人工设计，这种每一个神经元都与前一层所有神经元相连的神经网络也称全连接神经网络。



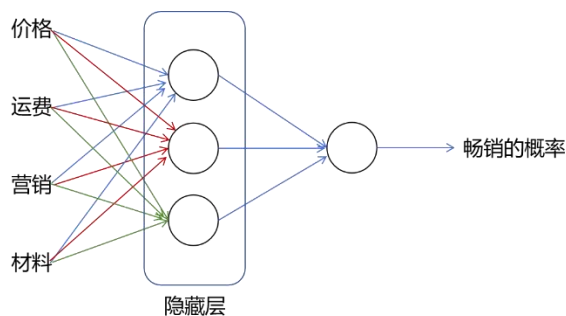
神经网络中自己学习隐藏特征的这一层叫做隐藏层，隐藏层能计算出输入数据中可能存在的一些隐藏关系，这些隐藏关系是原始数据没有直接告诉我们的。

如下图所示通常构建神经网络时有多个隐藏层，需要人工决定有多少个隐藏层，以及每层有多少个神经元，层数和每层的神经元个数的选择会对算法的性能产生影响。这种有多个隐藏层的神经网络也称为多层感知机（Multilayer Perceptron，MLP）。



4.2 单层神经网络

单层神经网络是指只有一个隐藏层的神经网络，仍然用前面的例子进行讲解。



第一层隐藏层有三个神经元，每一个神经元的激活值 a 由下式计算，

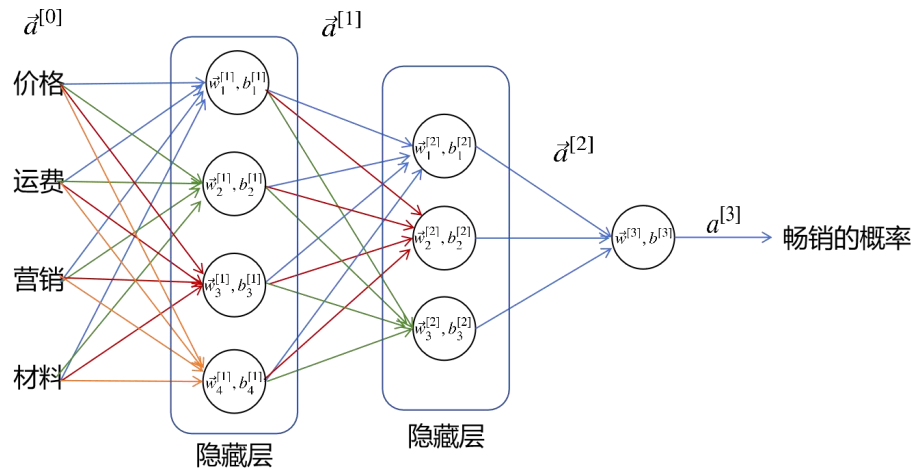
$$a = \frac{1}{1 + e^{-(\vec{w}x+b)}}$$

第一层第一个神经元的参数表示为 $\vec{w}_1^{[1]}, b_1^{[1]}$ ，其输出的激活值表示为 $a_1^{[1]}$ ，第一层第二个神经元的参数表示为 $\vec{w}_2^{[1]}, b_2^{[1]}$ ，其输出的激活值表示为 $a_2^{[1]}$ ，第一层第三个神经元的参数表示为 $\vec{w}_3^{[1]}, b_3^{[1]}$ ，其输出的激活值表示为 $a_3^{[1]}$ ，第一层的输出构

成一个向量 $\vec{d}^{[1]} = [a_1^{[1]}, a_2^{[1]}, a_3^{[1]}]$ 作为下一层的输入。第二层为输出层，因为是二分类问题，所以只有一个神经元，参数表示为 $\vec{w}^{[2]}, b^{[2]}$ ，输出值为 $\vec{d}^{[2]} = \frac{1}{1 + e^{-(\vec{w}^{[2]} \vec{d}^{[1]} + b^{[2]})}}$ 。

4.3 多层神经网络

下面来看具有两个隐藏层的神经网络如下图所示。



这是一个三层神经网络，不包括输入层，可以将输入层看成是第 0 层，所以三层神经网络包括两个隐藏层和一个输出层。第二层第一个神经元的输出为 $a_1^{[2]} = g(\vec{w}_1^{[2]} \vec{d}^{[1]} + b_1^{[2]})$ 这里 $g()$ 表示 sigmoid 函数，也称为激活函数（Activation Function），激活函数不仅可以用 sigmoid 函数，还有其他的激活函数用于替换这里的 sigmoid 函数。

题目： $a_2^{[2]} = ?$

- A. $a_2^{[2]} = g(\vec{w}_2^{[2]} \vec{d}^{[2]} + b_2^{[2]})$
- B. $a_2^{[2]} = g(\vec{w}_2^{[2]} \vec{d}^{[1]} + b_2^{[2]})$
- C. $a_2^{[2]} = g(\vec{w}_2^{[2]} \vec{d}_2^{[1]} + b_2^{[2]})$

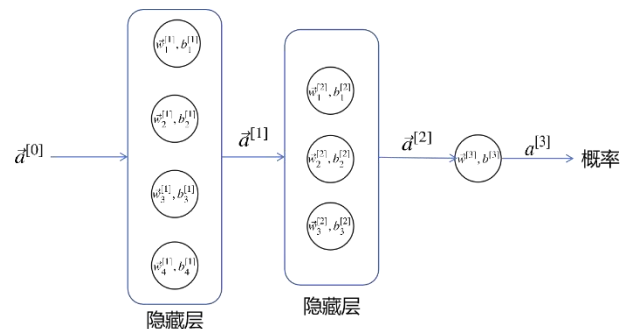
用 l 表示层数， j 表示第 j 个神经元， $a^{[0]}$ 表示输入特征向量，那么每个神经元的一般表示形式为：

$$a_j^{[l]} = g(\vec{w}_j^{[l]} \vec{d}^{[l-1]} + b_j^{[l]})$$

4.4 正向传播算法（Forward Propagation）

如下图所示在计算神经网络输出值的时候，从输入层开始从左向右依次进行计算，最后得到神经网络的输出，这个过程称为正向传播，用于神经网络的预测

阶段，以及训练时的正向传播阶段。运用正向传播算法，就可以使用已经训练好的神经网络参数进行预测，也称推理（Inference）。



下面把这个简单的例子推广到更一般的情况。假设神经网络的输入是 n 维向量 x ，输出是 k 维向量 y ，它实现了如下向量到向量的映射：

$$\mathbb{R}^n \rightarrow \mathbb{R}^k$$

把这个函数记为：

$$y = f(x)$$

用于分类问题时，比较输出向量中每个分量的大小，将最大值对应的分量下标作为分类结果。

神经网络第 l 层的计算写成矩阵和向量形式为：

$$\begin{aligned} z^{[l]} &= a^{[l-1]}W^{[l]} + b^{[l]} \\ a^{[l]} &= g(z^{[l]}) \end{aligned}$$

其中， $a^{[l-1]}$ 为第 $l-1$ 层的输出向量，也是第 l 层接收的输入向量； $W^{[l]}$ 是一个权重矩阵，维度 $s_{l-1} \times s_l$ ，其中， s_{l-1} 为上一层神经元的数量也就是上一层输出向量的元素数量， s_l 为本层神经元的数量， $W^{[l]}$ 的每一列为本层对应神经元的参数 $w_j^{[l]}$ ； $b^{[l]}$ 为本层的偏置向量，是一个有 s_l 个元素的行向量。在以后的符号表示中，我们用大写字母表示矩阵，小写字母表示向量或标量。激活函数作用于每一个神经元，产生输出向量。

假设神经网络有 L 层，正向传播算法的流程为：

设置 $a^{[0]} = x$

循环 $l = 1, 2, 3, \dots, L$ ，对每一层

计算 $z^{[l]} = a^{[l-1]}W^{[l]} + b^{[l]}$

计算 $a^{[l]} = g(z^{[l]})$

结束循环

输出向量 $a^{[L]}$ 作为神经网络的预测值

Part4-1 动手练

针对芯片是否保留的任务（数据集 ex2data2.txt），构建一个只有一个隐藏层的神经网络，隐藏层有 10 个神经元，加载已经训练好的参数 model.json 进行

预测。

手动构建模型并实现正向传播算法

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import json

# ===== 1 手动构建模型 =====
# 数据加载与预处理
def load_data(file_path):
    data = pd.read_csv(file_path, names=['x1', 'x2',
'accepted'])
    features = ['x1', 'x2']
    # z-score 标准化
    data[features] = (data[features] - data[features].mean())
/ data[features].std()
    cols = data.shape[1]
    X = data.iloc[:, 0:cols - 1].values
    y = data.iloc[:, cols - 1:cols].values.reshape(-1, 1)
    return X, y, data

# 神经网络模型
class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size):
        self.weights1 = np.random.randn(input_size,
hidden_size) * 0.01
        self.bias1 = np.zeros((1, hidden_size))
        self.weights2 = np.random.randn(hidden_size,
output_size) * 0.01
        self.bias2 = np.zeros((1, output_size))

    def _sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def forward(self, X):
        self.z1 = np.dot(X, self.weights1) + self.bias1
        self.a1 = self._sigmoid(self.z1)
        self.z2 = np.dot(self.a1, self.weights2) + self.bias2
        self.a2 = self._sigmoid(self.z2)
        return self.a2

    def predict(self, X, threshold=0.5):
        return (self.forward(X) > threshold).astype(int)
```

```
def load_model_from_json(self, filename):
    with open(filename, 'r') as f:
        model_data = json.load(f)
        self.weights1 = np.array(model_data["weights1"])
        self.bias1 = np.array(model_data["bias1"])
        self.weights2 = np.array(model_data["weights2"])
        self.bias2 = np.array(model_data["bias2"])
# ===== 1 手动构建模型=====

# 主程序
if __name__ == "__main__":
    # 参数设置
    input_size = 2 # 2 个特征
    hidden_size = 10 # 隐藏层神经元数量
    output_size = 1 # 二分类输出

    # 加载数据
    X, y, data = load_data('ex2data2.txt')
    # 初始化网络
    nn = NeuralNetwork(input_size, hidden_size, output_size)
    # 从 JSON 文件加载模型
    nn.load_model_from_json('model.json')

    # 预测与评估
    predictions = nn.predict(X)
    accuracy = np.mean(predictions == y)
    print(f'\nFinal Accuracy: {accuracy * 100:.2f}%')
```

4.5 反向传播算法

反向传播算法是神经网络的训练算法。回顾逻辑回归的训练过程如下：

$$\begin{aligned} &\text{repeat } \{ \\ &\quad w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b) \\ &\quad b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b) \\ &\} \end{aligned}$$

在逻辑回归中梯度的计算结果如下：

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$\frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})$$

在神经网络中代价函数依然表示为：

$$J(W, b) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \ln(f(x^{(i)})) + (1 - y^{(i)}) \ln(1 - f(x^{(i)}))]$$

这里 $f(x^{(i)})$ 是神经网络最后一层输出层的计算结果。神经网络训练过程中，优化目标函数的自变量是各层的权重矩阵 $W^{[l]}$ 和偏置向量 $b^{[l]}$ ，使用梯度下降法需要计算出代价函数对所有权重矩阵、偏置向量的梯度值，所以接下来的关键是这些梯度值的计算。反向传播算法是从最后一层输出层开始向前一层一层地计算梯度，这也是算法叫做反向传播算法的原因。

反向传播算法的迭代流程如下：

- (1) 正向传播，利用当前权重和偏置值，计算每一层的输出值 $a^{[l]}$ 。
- (2) 反向传播，计算输出层的误差项 $\delta^{[L]} = \hat{y} - y$ 。
- (3) 计算隐藏层各层的误差项：

$$\delta^{[l]} = (W^{[l+1]})^T (\delta^{[l+1]}) \cdot \frac{\partial}{\partial z^{[l]}} g(z^{[l]})$$

- (4) 计算损失函数对权重和偏置的梯度：

$$\begin{aligned} \nabla_{W^{[l]}} J(W, b) &= \delta^{[l]} (a^{[l-1]})^T \\ \nabla_{b^{[l]}} J(W, b) &= \delta^{[l]} \end{aligned}$$

- (5) 用梯度下降法更新权重和梯度：

$$\begin{aligned} W^{[l]} &= W^{[l]} - \alpha \nabla_{W^{[l]}} J(W, b) \\ b^{[l]} &= b^{[l]} - \alpha \nabla_{b^{[l]}} J(W, b) \end{aligned}$$

4.6 张量 (Tensor)

在 PyTorch 和 TensorFlow 这类深度学习框架中用张量表示数据，张量是一种数据结构，可以看成是多维数组。在 GPU 上运行时，张量能够高效地存储和执行矩阵计算。此外，PyTorch 和 TensorFlow 这类深度学习框架提供了自动梯度计算的功能，这对训练神经网络非常重要。

Part4-2 动手练

针对芯片是否保留的任务（数据集 ex2data2.txt），构建一个只有一个隐藏层的神经网络，隐藏层有 10 个神经元。

手动构建神经网络模型

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import json

# ===== 1 手动构建模型 =====
# 数据加载与预处理
def load_data(file_path):
    data = pd.read_csv(file_path, names=['x1', 'x2', 'accepted'])
    features = ['x1', 'x2']
    # z-score 标准化
    data[features] = (data[features] - data[features].mean()) / data[features].std()
    cols = data.shape[1]
    X = data.iloc[:, 0:cols - 1].values
    y = data.iloc[:, cols - 1:cols].values.reshape(-1, 1)
    return X, y, data

# 神经网络模型
class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size):
        self.weights1 = np.random.randn(input_size, hidden_size) * 0.01
        self.bias1 = np.zeros((1, hidden_size))
        self.weights2 = np.random.randn(hidden_size, output_size) * 0.01
        self.bias2 = np.zeros((1, output_size))

    def _sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def _sigmoid_derivative(self, g):
```

```

        return g * (1 - g)

    def forward(self, X):
        self.z1 = np.dot(X, self.weights1) + self.bias1
        self.a1 = self._sigmoid(self.z1)
        self.z2 = np.dot(self.a1, self.weights2) + self.bias2
        self.a2 = self._sigmoid(self.z2)
        return self.a2

    def backward(self, X, y, learning_rate=0.1):
        m = y.shape[0]
        # 输出层误差
        delta2 = self.a2 - y
        dW2 = np.dot(self.a1.T, delta2) / m
        db2 = np.sum(delta2, axis=0) / m
        # 隐藏层误差
        delta1 = np.dot(delta2, self.weights2.T) *
self._sigmoid_derivative(self.a1)
        dW1 = np.dot(X.T, delta1) / m
        db1 = np.sum(delta1, axis=0) / m
        # 参数更新
        self.weights2 -= learning_rate * dW2
        self.bias2 -= learning_rate * db2
        self.weights1 -= learning_rate * dW1
        self.bias1 -= learning_rate * db1

    def compute_loss(self, y):
        return -np.mean(y * np.log(self.a2) + (1 - y) * np.log(1
- self.a2))

    def predict(self, X, threshold=0.5):
        return (self.forward(X) > threshold).astype(int)

    def save_model_to_json(self, filename):
        model_data = {
            "weights1": self.weights1.tolist(),
            "bias1": self.bias1.tolist(),
            "weights2": self.weights2.tolist(),
            "bias2": self.bias2.tolist()
        }
        with open(filename, 'w') as f:
            json.dump(model_data, f)

    def load_model_from_json(self, filename):

```

```

        with open(filename, 'r') as f:
            model_data = json.load(f)
            self.weights1 = np.array(model_data["weights1"])
            self.bias1 = np.array(model_data["bias1"])
            self.weights2 = np.array(model_data["weights2"])
            self.bias2 = np.array(model_data["bias2"])

# 可视化决策边界
def plot_decision_boundary(X, y, nn, data):
    x1 = np.linspace(X[:, 0].min() - 0.5, X[:, 0].max() + 0.5,
50)
    x2 = np.linspace(X[:, 1].min() - 0.5, X[:, 1].max() + 0.5,
50)
    U, V = np.meshgrid(x1, x2)
    Z = np.zeros((len(x1), len(x2)))
    for i in range(len(x1)):
        for j in range(len(x2)):
            Z[i, j] = nn.predict(np.array([x1[i],
x2[j]])).item()

    positive = data[data['accepted'].isin([1])] # 筛选数据集中
    标签为 1 的数组
    negative = data[data['accepted'].isin([0])] # 筛选数据集中
    标签为 0 的数组
    plt.figure()
    plt.scatter(positive['x1'], positive['x2'], s=50, c='b',
marker='o', label='accepted')
    plt.scatter(negative['x1'], negative['x2'], s=50, c='r',
marker='x', label='not accepted')
    plt.contour(U, V, Z, levels=[0], linewidths=2, colors='g')
    plt.xlabel('x1')
    plt.ylabel('x2')
    plt.title('Decision Boundary')
    plt.legend()
    plt.show()

# 主程序
if __name__ == "__main__":
    # 超参数设置
    input_size = 2 # 2 个特征
    hidden_size = 10 # 隐藏层神经元数量
    output_size = 1 # 二分类输出
    learning_rate = 0.5

```

```

epochs = 5000

# 加载数据
X, y, data = load_data('ex2data2.txt')

# 初始化网络
nn = NeuralNetwork(input_size, hidden_size, output_size)

# 训练过程
loss_history = []
for epoch in range(epochs):
    # 前向传播
    output = nn.forward(X)
    # 计算损失
    loss = nn.compute_loss(y)
    loss_history.append(loss)
    # 反向传播
    nn.backward(X, y, learning_rate)
# 保存模型参数
nn.save_model_to_json('model.json')

# 从 JSON 文件加载模型
nn.load_model_from_json('model.json')

# 预测与评估
predictions = nn.predict(X)
accuracy = np.mean(predictions == y)
print(f'\nFinal Accuracy: {accuracy * 100:.2f}%')

# 绘制学习曲线
plt.plot(loss_history)
plt.xlabel('Epoch')
plt.ylabel('Cross-Entropy Loss')
plt.title('Training Loss Curve')
plt.show()

# 绘制决策边界
plot_decision_boundary(X, y, nn, data)

```

使用 PyTorch 库构建模型

安装 PyTorch 库, `pip install -i https://pytorch.tuna.tsinghua.edu.cn/simple torch`

```

import torch
import torch.nn as nn
import torch.optim as optim

```

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# ===== 2 使用 pytorch 库构建模型
=====

# 数据加载与预处理
def load_data(file_path):
    data = pd.read_csv(file_path, names=['x1', 'x2',
'accepted'])
    features = ['x1', 'x2']
    # z-score 标准化
    data[features] = (data[features] - data[features].mean())
/ data[features].std()
    X = data.iloc[:, 0:2].values
    y = data.iloc[:, 2:3].values
    return torch.tensor(X, dtype=torch.float32),
torch.tensor(y, dtype=torch.float32), data

# 神经网络模型
class NeuralNetwork(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(NeuralNetwork, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.fc2 = nn.Linear(hidden_size, output_size)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.sigmoid(self.fc1(x))
        x = self.sigmoid(self.fc2(x))
        return x

# 主程序
if __name__ == "__main__":
    # 超参数设置
    input_size = 2 # 2 个特征
    hidden_size = 10 # 隐藏层神经元数量
    output_size = 1 # 二分类输出
    learning_rate = 0.5
    lamda = 0.01
    epochs = 3000

    # 加载数据

```

```

X, y, data = load_data('ex2data2.txt')

# 初始化网络
model = NeuralNetwork(input_size, hidden_size, output_size)

# 损失函数和优化器
criterion = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=learning_rate)

# 训练过程
loss_history = []
for epoch in range(epochs):
    # 前向传播
    outputs = model.forward(X)
    loss = criterion(outputs, y)
    # L2 正则化
    reg = (lamda / (2 * y.shape[0])) *
sum(torch.linalg.norm(param) ** 2 for name, param in
model.named_parameters() if 'bias' not in name)
    loss = loss + reg
    loss_history.append(loss.item())
    # 反向传播
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

# 保存模型参数
torch.save({
    'model_state_dict': model.state_dict()
}, 'model.pth')

# 加载模型参数
checkpoint = torch.load('model.pth')
model.load_state_dict(checkpoint['model_state_dict'])

# 预测与评估
with torch.no_grad():
    predictions = model.forward(X).round()
    accuracy = (predictions == y).float().mean()
    print(f'\nFinal Accuracy: {accuracy * 100:.2f}%')

# 绘制学习曲线
plt.plot(loss_history)
plt.xlabel('Epoch')

```

```
plt.ylabel('Binary Cross-Entropy Loss')
plt.title('Training Loss Curve')
plt.show()
```

激活函数

到目前为止我们只使用了 **sigmoid** 激活函数，在神经网络中，可以使用其他激活函数使算法更强大。使用 **sigmoid** 函数激活值的取值范围只能是 0~1，如果需要更大的激活值可以选择其他激活函数，神经网络中常见的激活函数如下：

sigmoid	$f(z) = \frac{1}{1 + \exp(-z)}$
ReLU	$f(z) = \max(0, z)$
Softmax	$f(z) = \frac{\exp(z_i)}{\sum_{j=1}^k \exp(z_j)}$
Linear	$f(z) = z$

Softmax 回归

对于多分类问题，不再只是预测 $y=1$ 的概率，而是可能要预测 $y=1,2,3,4\dots$ 的概率，那么我们需要对多分类问题的标签有一个合适的数学表示。

one-hot 编码

假设我们现在需要对猫、狗、人这三个类别进行分类。现在需要一种数学表示方法，来表示猫、狗、人的分类。最容易想到的，便是以 0 代表猫，以 1 代表狗，以 2 代表人这种简单粗暴的方式，但这样做是不行的。因为分类标签一个重要的作用，就是要计算预测标签与真实标签之间的相似性，从而计算 **loss** 值，**loss** 值越小，说明预测标签与真实标签之间越接近。如果按照 0 代表猫，1 代表狗，2 代表人这种表示方法，那么猫和狗之间距离为 1，狗和人之间距离为 1，而猫和人之间距离为 2。这在参与损失计算的时候是完全不能接受的：互相独立的标签之间，竟然出现了 **loss** 不对等的情况。因此，需要有一种表示方法，将互相独立的标签表示为互相独立的数字，并且数字之间的距离也相等。

one-hot 也叫独热编码就是一种将离散的分类标签转换为二进制向量。**one-hot** 编码是一种向量表示，其中向量中只有一个元素的值是 1，其他所有元素都是 0。也可以用相反的方式表示，即向量中只有一个元素是 0，其他所有元素都是 1。下面是 **one-hot** 对类别进行编码的例子：

类别	one-hot 向量		
猫	1	0	0
狗	0	1	0
人	0	0	1

神经网络的输出层神经元个数与训练样本的类别数量一致，输出的值就是样

本属于某一分类的概率，比如当前的例子，某一个样本经过神经网络推理得到的输出值为有 70% 的概率是猫，20% 的概率是狗，10% 的概率是人，写成向量形式为 [0.7 0.2 0.1]，假设该样本的真实标签就是猫，猫的 one-hot 向量为 [1 0 0]，接下来就可以计算预测值与真实标签的损失。假设使用均方误差作为损失函数，那么损失值为：

$$loss = (0.7 - 1)^2 + (0.2 - 0)^2 + (0.1 - 0)^2 = 0.14$$

我们的优化目标是让 loss 的值最小。猫的得分是 0.7，而真实得分是 1，因此猫的得分还需要增大，狗和人的得分需要减小，计算完 loss 值后，反向传播调整权重，使猫的得分继续增大，而狗和人的得分继续减小。

Softmax 回归模型

Softmax 函数的形式如下：

$$f(z) = \frac{\exp(z_i)}{\sum_{j=1}^k \exp(z_j)}$$

与逻辑回归不同，Softmax 回归在解决多分类问题时需要使用 Softmax 函数对每一个类别的概率进行预测，因此一个样本属于每一类的概率计算如下：

$$\text{第一类: } z_1 = \vec{w}_1 \vec{x} + b_1, \quad a_1 = \frac{e^{z_1}}{\sum_{p=1}^N e^{z_p}}$$

$$\text{第二类: } z_2 = \vec{w}_2 \vec{x} + b_2, \quad a_2 = \frac{e^{z_2}}{\sum_{p=1}^N e^{z_p}}$$

...

$$\text{第 } N \text{ 类: } z_N = \vec{w}_N \vec{x} + b_N, \quad a_N = \frac{e^{z_N}}{\sum_{p=1}^N e^{z_p}}$$

这种计算方式保证了概率统计中概率之和为 1 的条件。写成一般形式为：

$$z_k = \vec{w}_k \vec{x} + b_k$$

$$a_k = \frac{e^{z_k}}{\sum_{p=1}^N e^{z_p}}$$

$$k = 1, 2, \dots, N$$

下面是逻辑回归的损失函数，

$$L(f(x^{(i)}), y^{(i)}) = \begin{cases} -\ln(f(x^{(i)})), & \text{if } y^{(i)} = 1 \\ -\ln(1 - f(x^{(i)})), & \text{if } y^{(i)} = 0 \end{cases}$$

推广到 Softmax 回归多分类问题，可以得到如下损失函数：

$$L(f(x^{(i)}), y^{(i)}) = \begin{cases} -\ln(a_1), & \text{if } y^{(i)} = 1 \\ -\ln(a_2), & \text{if } y^{(i)} = 2 \\ \dots \\ -\ln(a_N), & \text{if } y^{(i)} = N \end{cases}$$

将类别标签 $y^{(i)}$ 转换成 one-hot 向量 $y^{(i)} = [1, 0, 0, \dots, 0]$, 将 Softmax 计算的样本属于每一类的概率也写成向量形式 $f(x^{(i)}) = [a_1, a_2, \dots, a_N]$, 则 Softmax 回归的损失函数可以写成一般形式:

$$L(f(x^{(i)}), y^{(i)}) = -y^{(i)} \cdot \ln(f(x^{(i)}))$$

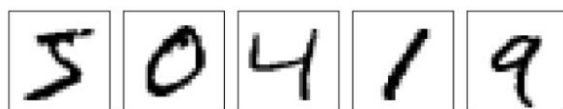
Softmax 回归的代价函数为:

$$J(W, \vec{b}) = -\frac{1}{m} \sum_{i=1}^m y^{(i)} \cdot \ln(f(x^{(i)}))$$

在将神经网络应用于多分类问题时, Softmax 回归模型可以看作是神经网络的输出层。因此用于多分类问题的神经网络通常将 Softmax 函数作为输出层的激活函数。

Part4-3 动手练

MNIST 数据集是美国国家标准与技术研究院收集整理的大型手写数字数据库, 包含 60,000 个示例的训练集以及 10,000 个示例的测试集。其中的图像的尺寸为 28*28。采样数据显示如下:



MNIST 图像文件有一个特定的文件头, 其中包含一些元数据, 这个文件头由 16 字节组成, 它们分别表示:

magic_number: 一个魔数, 用于标识文件格式。对于图像文件, 这个值通常是 0x00000803。

num_images: 文件中的图像数量。

num_rows: 每张图像的行数 (即高度)。28

num_cols: 每张图像的列数 (即宽度)。28

训练数据可视化

```
import numpy as np
import matplotlib.pyplot as plt
import json
import gzip

# ===== 1 数据可视化 =====
def load_images(filename):
```

```

    with gzip.open(filename, 'rb') as f:
        magic_number = int.from_bytes(f.read(4), 'big')
        num_images = int.from_bytes(f.read(4), 'big')
        num_rows = int.from_bytes(f.read(4), 'big')
        num_cols = int.from_bytes(f.read(4), 'big')
        images = np.frombuffer(f.read(),
dtype=np.uint8).reshape(num_images, num_rows, num_cols)
        return images

def load_labels(filename):
    with gzip.open(filename, 'rb') as f:
        magic_number = int.from_bytes(f.read(4), 'big')
        num_labels = int.from_bytes(f.read(4), 'big')
        labels = np.frombuffer(f.read(), dtype=np.uint8)
        return labels

# 定义文件路径
train_images_file = './MNIST/raw/train-images-idx3-ubyte.gz'
train_labels_file = './MNIST/raw/train-labels-idx1-ubyte.gz'

# 加载图像和标签数据
train_images = load_images(train_images_file)
train_labels = load_labels(train_labels_file)

# 显示图像
num_images_to_display = 5
for i in range(num_images_to_display):
    plt.imshow(train_images[i], cmap='gray')
    plt.title(f"Label: {train_labels[i]}")
    plt.axis('off')
    plt.show()

```

手动构建一个只有一个隐藏层的神经网络，隐藏层神经元个数为 128。

```

# 加载测试集
test_images_file = './MNIST/raw/t10k-images-idx3-ubyte.gz'
test_labels_file = './MNIST/raw/t10k-labels-idx1-ubyte.gz'
test_images = load_images(test_images_file)
test_labels = load_labels(test_labels_file)

# 将图像数据转换为一维向量，并归一化
X_train = train_images.reshape(train_images.shape[0], -1) / 255.0
X_test = test_images.reshape(test_images.shape[0], -1) / 255.0

# 将标签进行 one-hot 编码
num_classes = 10
y_train = np.eye(num_classes)[train_labels]
y_test = np.eye(num_classes)[test_labels]

```

```

class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size):
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size

        # 初始化权重
        self.W1 = np.random.randn(self.input_size,
self.hidden_size)
        self.b1 = np.zeros((1, self.hidden_size))
        self.W2 = np.random.randn(self.hidden_size,
self.output_size)
        self.b2 = np.zeros((1, self.output_size))

    def forward(self, X):
        # 前向传播
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = np.maximum(0, self.z1) # ReLU 激活函数
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        exp_scores = np.exp(self.z2) # softmax
        self.probs = exp_scores / np.sum(exp_scores, axis=1,
keepdims=True)
        return self.probs

    def backward(self, X, y, learning_rate=0.01):
        # 反向传播
        m = X.shape[0]
        delta2 = self.probs - y
        dW2 = np.dot(self.a1.T, delta2) / m
        db2 = np.sum(delta2, axis=0, keepdims=True) / m
        delta1 = np.dot(delta2, self.W2.T) * (self.a1 > 0)
        dW1 = np.dot(X.T, delta1) / m
        db1 = np.sum(delta1, axis=0, keepdims=True) / m

        # 更新权重
        self.W1 -= learning_rate * dW1
        self.b1 -= learning_rate * db1
        self.W2 -= learning_rate * dW2
        self.b2 -= learning_rate * db2

    # 梯度下降法训练
    def train(self, X, y, num_epochs=1000, learning_rate=0.01):
        for epoch in range(num_epochs):

```

```

        # 前向传播和反向传播
        probs = self.forward(X)
        self.backward(X, y, learning_rate)

        if epoch % 100 == 0:
            loss = self.calculate_loss(X, y)
            print(f"Epoch {epoch}, Loss: {loss}")

    def predict(self, X):
        # 预测类别
        return np.argmax(self.forward(X), axis=1)

    def calculate_loss(self, X, y):
        # 计算交叉熵损失
        log_probs = -np.log(self.probs[range(X.shape[0]),
np.argmax(y, axis=1)])
        return np.mean(log_probs)

# 定义网络结构和参数
input_size = 784 # 28*28 像素
hidden_size = 128
output_size = 10

# 创建神经网络实例
nn = NeuralNetwork(input_size, hidden_size, output_size)

# 训练模型
nn.train(X_train, y_train, num_epochs=10, learning_rate=0.01)
# 可选（使用梯度下降法进行训练）

# 预测并评估模型
train_predictions = nn.predict(X_train)
test_predictions = nn.predict(X_test)

train_accuracy = np.mean(train_predictions == train_labels)
test_accuracy = np.mean(test_predictions == test_labels)

print(f"Train Accuracy: {train_accuracy:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")

```

作业：使用 PyTorch 库构建神经网络进行手写数字识别